Reporte

Nombre: Emiliano Sánchez Juárez Valeria Nohemí Cisneros Moreno	**Matrícula**: 2169669 2178081
Nombre del curso: Componentes de Lógica de Negocio	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 24	**Actividad**: Diseño Arquitectural del Sistema: Architecture Decision Record
Fecha: 10/12/2026	
Bibliografía (Formato APA)	

Objetivo:

En esta actividad convertirán el análisis de negocio elaborado en la Unidad 2 en una propuesta de arquitectura de software. Trabajarán con la misma empresa y el mismo caso. La inteligencia artificial podrá apoyar el diseño y la revisión, pero cada decisión deberá justificarse con evidencia del proyecto.

ADR-01	
Título	Selección del patrón de lógica de negocio Domain Model
Estado	Aceptada
Contexto	PetMarket requiere administrar un proceso de pedidos que involucra distintas reglas de negocio y la interacción entre Pedido, Línea de pedido, Almacén, Inventario, Pago, Envío y Reembolso. Además, existen diferentes estados y condiciones que deben respetarse, como confirmar un pedido únicamente cuando el pago ha sido aprobado, consolidar los pedidos divididos y evitar cancelaciones después del envío. Dentro del caso se identificaron 14 reglas de negocio (RN-01 a RN-14), así como cuatro casos de uso principales (CU-01 a CU-04). Esta complejidad hace necesario centralizar las reglas y proteger la consistencia del proceso.
Alternativas	• **Transaction Script:** implementa cada caso de uso mediante procedimientos independientes. Es sencillo al inicio, pero puede duplicar reglas y dificultar el mantenimiento. • **Table Module:** organiza la lógica por tablas o grupos de datos. Facilita ciertas operaciones, pero las reglas que involucran varios conceptos pueden quedar dispersas. • **Domain Model:** representa lo conceptos del negocio mediante objetos que contienen sus datos, comportamientos y reglas. Requiere mayor inversión inicial, pero facilita el control de estados y la evolución del sistema.
Decisión	Se selecciona Domain Model como patrón principal, aplicado al núcleo del proceso de pedidos. Las entidades y agregados del dominio serán responsables de mantener sus propias reglas e invariantes. Se utilizará Transaction Script únicamente para operaciones sencillas que no requieran reglas de negocio complejas, como algunas notificaciones y consultas simples.
Consecuencias	• *Beneficios:* centralización de reglas, mayor consistencia de los pedidos y facilidad para incorporar nuevos almacenes o políticas de negocio. • *Costos:* mayor tiempo de diseño, necesidad de definir correctamente las entidades y agregados y trabajo adicional para mapear el dominio a la base de datos. • *Riesgos:* un diseño demasiado complejo si se aplica el patrón a operaciones sencillas. • *Trabajo posterior:* diseñar los agregados Pedido, Almacén, Pago, Envío y Reembolso; definir sus responsabilidades, invariantes y eventos; y comprobar su consistencia con los diagramas.
Trazabilidad	• Casos de uso: CU-01, CU-02, CU-03 y CU-04. • Reglas de negocio: RN-01 a RN-14. • BPMN del proceso de pedidos, glosario del dominio y reglas relacionadas con inventario, pago, pedidos divididos y cancelaciones.

ADR-02	
Título	Implementación de una Service Layer para coordinar los casos de uso.
Estado	Propuesta
Contexto	PetMarket necesita ejecutar operaciones como iniciar el checkout, procesar pagos, preparar pedidos y registrar cancelaciones. Estas operaciones requieren coordinar varios componentes, incluyendo repositorios, inventario, pagos y objetos del dominio. Si toda la lógica se concentra en los servicios de aplicación, las reglas podrían duplicarse y sería más difícil mantener una separación clara entre la coordinación del proceso y las decisiones propias del negocio.
Alternativas	• **Concentrar toda la lógica en los servicios:** simplifica inicialmente la coordinación, pero genera servicios con demasiadas responsabilidades. • **Distribuir la lógica entre controladores y repositorios:** puede mezclar la comunicación con el usuario, las reglas de negocio y la persistencia. • **Utilizar Service Layer:** los servicios coordinan las operaciones y delegan las reglas de negocio a los objetos del dominio. Esto requiere definir responsabilidades e interfaces, pero mejora la organización.
Decisión	Se propone implementar una Service Layer con servicios de aplicación específicos para cada caso de uso: • *CheckoutService*: coordina la validación del carrito y la disponibilidad del inventario. • *PagoService*: coordina el procesamiento del pago y comunica su resultado. • *PreparacionPedidoService*: coordina la preparación de líneas, la consolidación y la actualización del pedido. • *CancelacionService*: coordina la cancelación y el proceso de reembolso correspondiente.
Consecuencias	• *Beneficios:* responsabilidades mejor definidas, menor duplicación de reglas y mayor facilidad para probar cada caso de uso. • *Costos:* creación de servicios, interfaces y métodos específicos para las operaciones del negocio. • *Riesgos:* que los servicios terminan concentrando reglas que deberían pertenecer al dominio. • *Trabajo posterior:* definir las entradas y salidas de cada servicio, sus dependencias, los límites transaccionales y el manejo de errores.

ADR-02	
Trazabilidad	• Casos de uso: CU-01, CU-02, CU-03 y CU-04. • Reglas de negocio: RN-01 a RN-14. • Componentes relacionados: *CheckoutService, PagoService, PreparacionPedidoService, CancelacionService, PedidoRepository* e *InventarioAlmacenRepository.* • Flujo BPMN del pedido y sus excepciones de pago, inventario, preparación y cancelación.

ADR-03	
Título	Separación de Commands y Queries mediante CQRS básico.
Estado	Propuesta
Contexto	PetMarket necesita realizar operaciones que modifican el estado de los pedidos, como iniciar un checkout, procesar un pago, preparar una línea o cancelar una compra. También requiere consultar información sin modificar el estado del sistema, como el estado de un pedido o los reportes de cancelaciones y devoluciones. Si ambas responsabilidades se mezclan, las consultas podrían quedar acopladas a la lógica de modificación y resultar más difíciles de mantener. Por ello, se considera conveniente distinguir las operaciones que modifican datos de aquellas que únicamente recuperan información.
Alternativas	• **Utilizar los mismos métodos para escritura y consulta:** reduce la cantidad inicial de componentes, pero mezcla responsabilidades. • **Separar Commands y Queries con CQRS básico:** permite definir contratos, manejadores y resultados específicos para cada tipo de operación, aunque requiere componentes adicionales. • **Implementar CQRS con bases de datos separadas:** ofrece una separación mayor, pero incrementa la complejidad de infraestructura y sincronización.

ADR-03	
Decisión	Se propone implementar CQRS básico sin separar las bases de datos. Los Commands modificarán el estado del sistema, mientras que las Queries devolverán información sin realizar cambios. Se contemplan los siguientes ejemplos: <u>Commands</u> ● *IniciarCheckoutCommand:* inicia el checkout y permite crear el pedido. ● *ProcesarPagoCommand:* procesa el pago y registra su resultado. ● *CancelarPedidoCommand:* solicita la cancelación y coordina el reembolso cuando corresponda. ● *MarcarLineaPreparadaCommand:* registra la preparación de una línea del pedido. <u>Queries</u> ● *ConsultarEstadoDelPedidoQuery:* obtiene el estado actual del pedido. ● *ConsultarCancelacionesYDevolucionesQuery:* genera un reporte a partir de un rango de fechas.
Consecuencias	● *<u>Beneficios:</u>* separación clara de responsabilidades, consultas más fáciles de adaptar a las necesidades de lectura y mayor claridad en las operaciones que modifican el estado. ● *<u>Costos:</u>* creación de Commands, Queries, Handlers y modelos de lectura específicos. ● *<u>Riesgos:</u>* duplicación innecesaria de estructuras si la separación se aplica a operaciones demasiado simples. ● *<u>Trabajo posterior:</u>* definir los contratos de entrada y salida, implementar los Handlers y verificar que las consultas no modifiquen información del dominio.
Trazabilidad	● Casos de uso: CU-01, CU-02, CU-03 y CU-04. ● Reglas de negocio: RN-01 a RN-14, según la operación ejecutada. ● Operaciones relacionadas: *IniciarCheckoutCommand, ProcesarPagoCommand, CancelarPedidoCommand, MarcarLineaPreparadaCommand, ConsultarEstadoDelPedidoQuery* y *ConsultarCancelacionesYDevolucionesQuery*. ● Flujo del proceso de pedidos y necesidades de seguimiento del estado y consulta de cancelaciones y devoluciones.